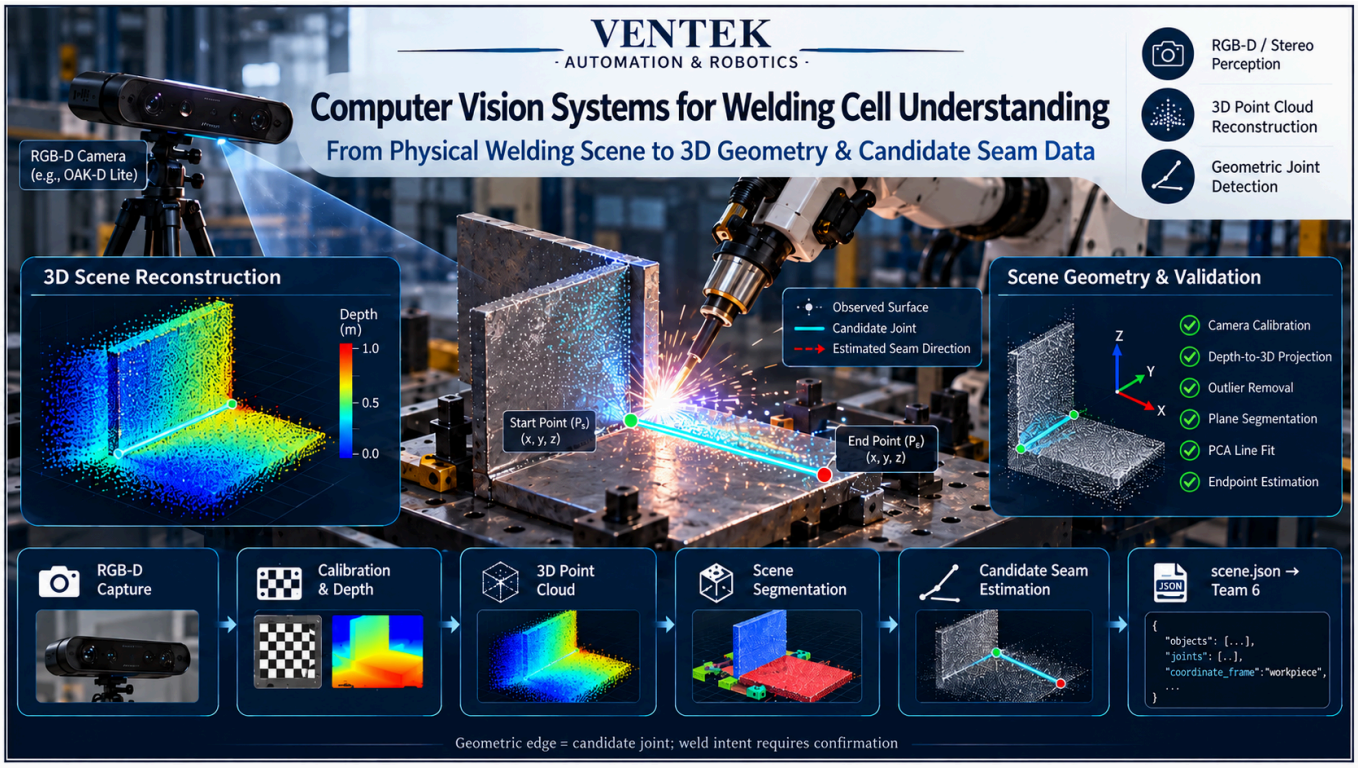
DOMAIN 5 | TEAM 5 TECHNICAL PROPOSAL



Computer Vision and Weld Intelligence

Purpose: convert calibrated sensor data into a 3D description of workpieces, fixtures and candidate weld-joint geometry for Team 6 path planning.

Document status: proposed baseline and implementation plan. The example figures and JSON values are illustrative; no physical scan, camera accuracy or detection performance is claimed as verified.

1. Objective, scope and sensing choice

Core problem. Given a physical welding assembly, Team 5 must reconstruct enough 3D scene geometry to locate workpieces and fixtures, propose candidate joint/seam geometry, estimate supported parameters, and provide a documented data contract to Team 6. The output supports path planning; it does not command the robot or certify weld quality.

Current scope versus future scope

Current scope (baseline deliverable)	Future scope / explicitly not claimed
Acquire/import RGB-D or scanner data; validate calibration and data quality.	Arc-on, real-time seam tracking during welding; high-speed sensor/robot synchronization.
Point-cloud construction, denoising, plane/cluster segmentation of visible workpieces, fixtures and clamps.	Deep-learning segmentation or neural weld recognition unless the geometric baseline and labelled data justify it.
Generate candidate joint points; estimate endpoints, length, direction and optional normals when supported.	Weld-quality inspection (cracks, porosity, bead quality) and weld-intent certification.
Export candidate geometry with frame, units, timestamp, confidence and validity to scene.json.	Robot trajectory execution, robot safety approval and industrial tolerance certification.

Sensing-method comparison

Method	What it provides	Trade-off for this project	Decision
RGB only	Colour/texture and 2D edges.	Single RGB view does not provide reliable metric 3D coordinates without extra geometry, multiple calibrated views or a known model.	Fallback only.
Stereo RGB-D / depth camera	RGB plus dense metric depth for scene-level point cloud.	Good baseline for 3D geometry; accuracy depends on range, texture, calibration, ambient light and surface reflectivity.	Preferred baseline, subject to bench test.
Structured-light 3D	Depth from a projected pattern and camera observation.	May give useful fixed-range detail but needs dedicated hardware/setup and surface/lighting validation.	Alternative if available.
Laser-line profile sensor	High-resolution local cross-section of a seam.	Can suit seam-following, but usually needs scanning motion, calibrated sensor-to-part geometry and more integration.	Future option if stereo depth fails requirements.

Selection rationale: start with calibrated stereo RGB-D because the target is scene reconstruction plus candidate-joint geometry, not only a single seam profile. First use hardware already provided by the project. Do not buy a sensor until the actual working distance, smallest feature size, assembly size and required error tolerance are known.

2. Proposed workflow and acquisition conditions

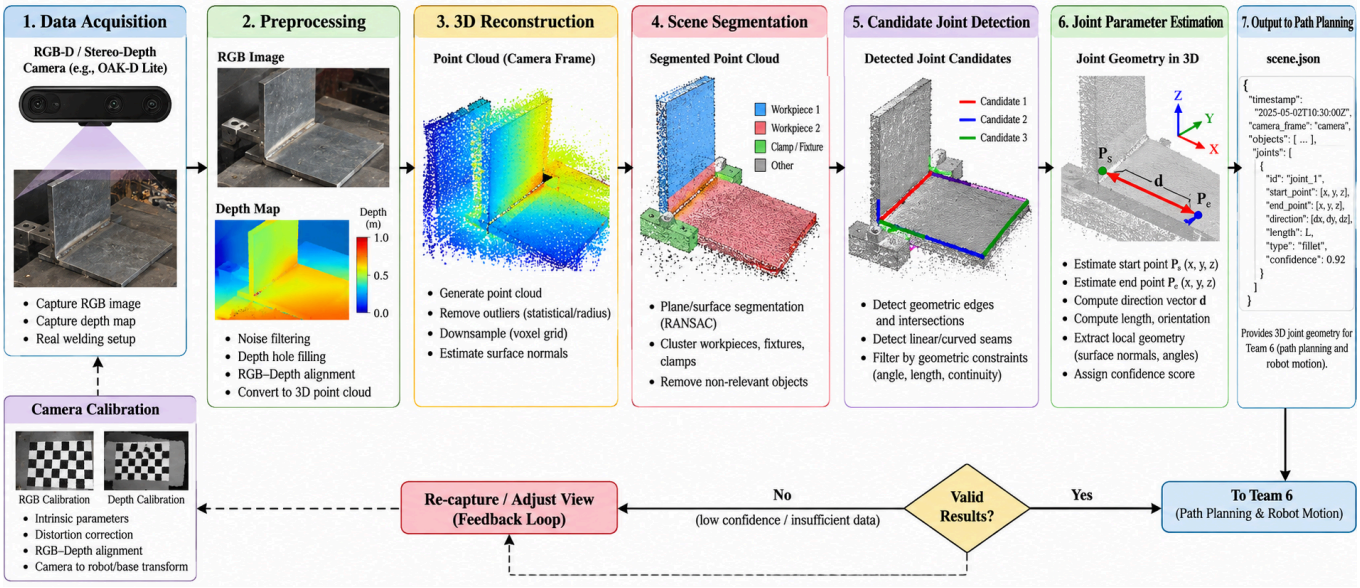


Figure 1. Team 5 end-to-end workflow. It is a proposed architecture; embedded sample imagery is illustrative, not a scan result from the project assembly.

Stage	Method / output	Check before continuing
1. Data acquisition	Capture synchronized RGB and depth or import a 3D scanner cloud; save source files and metadata.	Stable mount; correct focus/exposure; timestamp; no missing input files.
2. Preprocessing	Depth/RGB alignment where supported, invalid-depth mask, conservative noise filtering.	Check alignment and valid-depth coverage; avoid filling holes that invent seam geometry.
3. 3D reconstruction	Project depth pixels into camera-frame XYZ using calibrated intrinsics and depth scale.	Units and camera optical-axis convention documented.
4. Scene segmentation	RANSAC plane fitting and DBSCAN clustering for large surfaces/objects [1].	Compare clusters against annotated assembly; do not assign semantic labels from geometry alone.
5. Candidate joint detection	Use nearby non-coplanar surface intersections, visible edges/gaps and geometric constraints.	Keep candidate and confirmed weld intent as separate states.
6. Parameter estimation	PCA line direction and projected endpoints; compute length and quality indicators.	Reject too-short, sparse, high-residual or poorly supported candidates.
7. Output to Team 6	Write schema-versioned scene.json with units, frame, timestamp, confidence and validity.	Validate good and bad payloads; agree interface before integration.

Required data and calibration

- RGB image and depth map (or scanner XYZ cloud), camera intrinsics, depth scale, capture timestamp, and camera/frame identifier.
- A measured camera-to-assembly/world transform if output is to be expressed in assembly_world; a further calibrated transform is required before claiming robot_base coordinates.

- Reference dimensions/part labels from the physical assembly, drawing or operator annotations for evaluation and weld-intent review.
- Working distance is currently not specified. Test at the actual proposed distance and multiple views; do not assume a datasheet accuracy transfers to reflective steel or the full workcell.
- Initial baseline assumes a static assembly captured before arc-on welding, indoor lighting, a rigid camera mount and visible joint surfaces. Arc glare, occlusion and reflective metal are separate stress cases.

3. Baseline algorithm and geometric parameter estimation

The first implementation uses classical geometry rather than a neural detector. Open3D provides point-cloud plane segmentation with RANSAC and density-based DBSCAN clustering [1]. OpenCV provides camera calibration tools [2]. These capabilities are supported by library documentation; their performance on this assembly remains to be measured.

3.1 Depth-to-3D projection

For pixel (u, v), depth value z, and calibrated intrinsics (f_x, f_γ, c_x, c_γ), convert a valid depth pixel to camera-frame coordinates:

```
X_c = (u - c_x) * Z_c / f_x
Y_c = (v - c_y) * Z_c / f_y
Z_c = depth_value * depth_scale_to_metres
```

Apply a calibrated homogeneous transform only when available: p_assembly = T_assembly_camera · p_camera. Document whether transforms use column vectors and the meaning of T_target_source. Do not invent extrinsics.

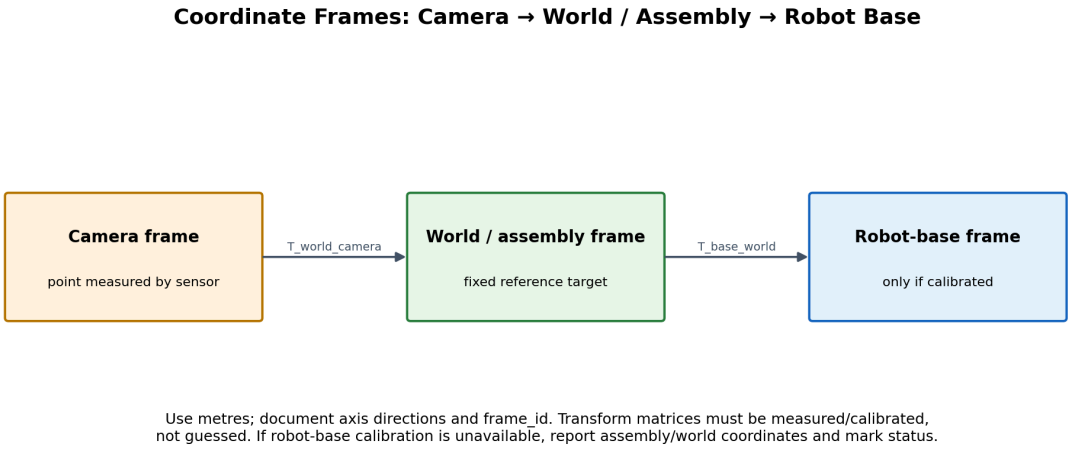


Figure 3. Coordinate-frame concept. Camera-to-assembly and assembly-to-robot transforms must come from calibration.

3.2 Point-cloud processing and candidate generation

1. Crop to the assembly region of interest, remove invalid points and apply conservative voxel downsampling/outlier filtering.
2. Fit dominant planar surfaces with RANSAC; cluster spatially separate components with DBSCAN [1].
3. Generate candidate joints from supported intersections/edges between adjacent surfaces, using angle, separation, continuity and minimum support checks.
4. Filter candidates, fit a PCA line to candidate 3D points, estimate endpoints and attach a heuristic score based on support, residual and depth quality.

5. Emit a candidate only when data is sufficient; otherwise report LOW_CONFIDENCE or INVALID_INPUT and request recapture/review.

3.3 PCA line fit, endpoints, length and direction

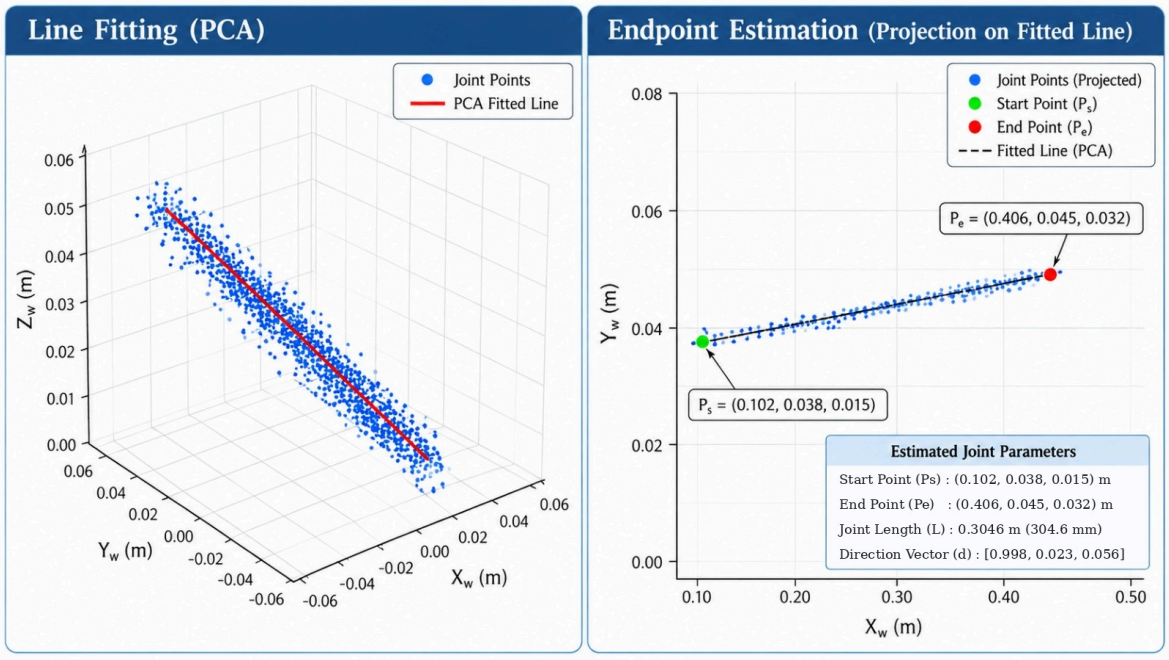


Figure 2. PCA-based line fitting and projected endpoint estimation. The plotted points and example coordinates are illustrative, not measured project data.

Given N candidate points $p_i = [x_i, y_i, z_i]^T$, compute their centroid $\mu = (1/N) \Sigma p_i$ and centered points $q_i = p_i - \mu$. The sample covariance matrix is $C = (1/(N-1)) \Sigma q_i q_i^T$. Find eigenpairs $C v_\square = \lambda_\square v_\square$; the eigenvector v_1 associated with the largest eigenvalue gives the dominant line direction.

Fitted line: $\ell(t) = \mu + t\, v_1$
Point projection: $t_i = v_1^T (p_i - \mu)$
Endpoint scalars: $t_{min} = \min(t_i),\ t_{max} = \max(t_i)$
Endpoints: $P_s = \mu + t_{min} v_1,\ P_e = \mu + t_{max} v_1$
Length: $L = \|P_e - P_s\|$
Unit direction: $\hat{d} = (P_e - P_s) / \|P_e - P_s\|$

Example from the illustration: $P_\square = (0.102, 0.038, 0.015)$ m and $P_e = (0.406, 0.045, 0.032)$ m. Thus $\Delta P = (0.304, 0.007, 0.017)$ m, $L \approx 0.3046$ m (304.6 mm), and $\hat{d} \approx (0.998, 0.023, 0.056)$. These values are a worked example only. Reject outliers before PCA or use robust projection percentiles; raw min/max can overestimate length.

Critical limitation: PCA fits a line to points already marked as a candidate. It does not decide whether an edge should be welded. Use drawings, assembly instructions or operator confirmation where available; otherwise set weld_intent_confirmed=false.

4. Input/output contracts and team interfaces

4.1 Input example: capture_manifest.json

The manifest identifies the capture files and the metadata needed by the pipeline; the image/depth pixels remain in separate files. Every value below is illustrative and must be populated from the actual sensor and calibration result.

```
{
 "schema_version": "0.1",
 "capture_id": "capture_0001",
 "timestamp_s": 1792300000.0,
 "frame_id": "camera_optical",
 "sensor": {"source_type": "stereo_rgbd", "model": "TBD_actual_device"},
 "rgb_file": "captures/capture_0001_rgb.png",
 "depth_file": "captures/capture_0001_depth.png",
 "depth_scale_m_per_unit": 0.001,
 "intrinsics_px": {"fx": 700.0, "fy": 700.0, "cx": 640.0, "cy": 360.0},
 "T_assembly_camera": null,
 "calibration_status": "intrinsics_available_extrinsics_pending"
}
```


4.2 Output example: scene.json


```
{
 "schema_version": "0.1",
 "timestamp_s": 1792300000.0,
 "frame_id": "assembly_world",
 "units": {"position": "m", "angle": "rad"},
 "calibration_status": "example_only_not_verified",
 "parts": [{"part_id": "part_001", "type": "workpiece", "confidence": 0.91, "valid": true}],
 "candidate_joints": [{
  "joint_id": "candidate_001",
  "part_ids": ["part_001", "part_002"],
  "start_point_m": [0.102, 0.038, 0.015],
  "end_point_m": [0.406, 0.045, 0.032],
  "direction_unit": [0.998, 0.023, 0.056],
  "length_m": 0.3046,
  "local_normal": null,
  "confidence": 0.72,
  "valid": true,
  "weld_intent_confirmed": false,
  "status": "candidate_requires_confirmation"
 }],
 "warnings": ["Mock values; not measured data"]
}
```


4.3 Field meanings and conventions

Field	Meaning / rule
timestamp_s	UTC Unix epoch seconds at capture; not a formatted date string.
frame_id	Frame in which all points are expressed. Use assembly_world only after the camera-to-world transform is calibrated; use robot_base only after its transform is agreed and measured.
position/length	Metres. Raw depth units and depth_scale_m_per_unit must be recorded in the input manifest.
direction_unit	Normalized 3D vector from start to end; dimensionless. Verify norm ≈ 1.
confidence	0–1 heuristic quality score based on available evidence; not a calibrated probability until evaluated on labelled data.

Field	Meaning / rule
valid/status	Indicates whether current geometric/data checks passed; distinguish OK, LOW_CONFIDENCE, INVALID_INPUT and NO_CANDIDATE as agreed.
weld_intent_confirmed	False by default. Geometry alone cannot confirm the manufacturing instruction to weld this edge.

4.4 Interfaces and ownership

Provider / consumer	Contract or dependency	Agreement required
Physical assembly / sensor owner → Team 5	Assembly access, RGB/depth or point cloud, sensor model and calibration metadata.	Working distance, mount, light, safety, reference dimensions and file access.
Team 5 → Team 6 path planning	scene.json: candidate endpoints/waypoints and geometric descriptors.	Schema version, timestamp convention, frame_id, units, required fields, stale/invalid behavior and coordinate transform.
Robotics/integration owner ↔ Team 5	Camera-to-assembly and possibly assembly-to-robot-base calibration.	Owner and procedure for T_assembly_camera and T_robotbase_assembly; never infer from camera pose alone.
Assembly/quality owner ↔ Team 5	Part labels, drawings or operator judgement for candidate confirmation.	Definition of weld-intent ground truth and labelled test examples.

Team 6's sample interface uses position in metres and illustrates quaternion_xyzw for pose orientation; this report does not emit a tool orientation unless the application supplies enough local geometry and the teams agree the convention. The JSON above is a proposal, not an approved contract.

5. Exact baseline technology stack and hardware recommendation

5.1 Proposed software stack

Component	Baseline choice	Why it is needed / condition
Language/runtime	Python 3.11	Core perception logic, scripts, repeatable experiments. Pin package versions in requirements.txt after the chosen sensor smoke test passes.
Image calibration	OpenCV 4.x (calib3d; ArUco/ChArUco if a board is available) [2]	Lens calibration, image handling, marker-based reference poses; camera-to-assembly transform must be measured.
Sensor capture	DepthAI Python SDK for OAK-D Lite, OR pyrealsense2/librealsense for RealSense	Use only the SDK matching the actual camera. Do not install both unless comparing devices.
Point-cloud processing	Open3D 0.19+; NumPy; SciPy [1]	Load/build clouds, filtering, voxel downsampling, RANSAC planes, DBSCAN, distances, PCA and visualization.

Component	Baseline choice	Why it is needed / condition
Interface / testing	jsonschema 4.x and pytest 8.x	Validate scene.json/capture manifest and add repeatable unit/integration tests.
Optional tools	CloudCompare for offline inspection; ROS 2/RViz2 only if the integrated project needs ROS; Gazebo only for downstream robot simulation.	Not required to generate a point cloud or scene.json directly.

Versions above are the proposed baseline stack, not a claim that the packages are already installed or tested together. First create an environment and run a small import/capture smoke test; then freeze exact patch versions in a lock/requirements file.

5.2 Sensor/hardware shortlist and indicative cost

Candidate	Manufacturer range/specification	Observed indicative price in India	Fit / caveat
Luxonis OAK-D Lite	Vendor lists ideal range about 0.8–12 m; reported median depth error below 2% at <4 m in vendor conditions [3].	About ₹33,899–₹36,749 in reseller listings; availability varies [4][5]. Manufacturer shop listed US$269 [6].	Candidate for scene-wide capture at the correct standoff. Do not assume performance at close range or on reflective weld metal.
Intel RealSense D405	Official ideal range 7–50 cm; official product page lists ±2% depth accuracy at 50 cm [7].	About ₹29,199–₹34,999 in Indian listings; some listings may be backorder/out of stock [8][9].	Better spec match for close-range detail; may not cover the whole assembly from one view.

Recommendation: use hardware already available first. If none exists, measure the planned working distance and smallest relevant seam/feature, then borrow or trial the best-matched sensor. OAK-D Lite is the initial scene-wide candidate only when the mount distance and field of view suit its range; consider D405 when the camera must operate close to the workpiece. Prices above are listing snapshots accessed 18 Oct 2026, not purchase quotes. Confirm stock, GST, delivery, warranty and calibration support before any purchase.

Do not purchase solely from datasheet claims. Bench-test the actual surface material, glare, lighting, standoff, occlusion and endpoint error against a measured reference. A published comparison of several stereo cameras for tabletop objects also emphasizes that performance varies by scene and distance; it does not validate our exact camera/workcell [10].

6. Validation plan, metrics and known limitations

6.1 Experiment and test matrix

Acquire at least three repeated captures of the same physical assembly from the planned fixed standoff after calibration. Measure several accessible reference dimensions using a suitable physical reference (e.g., calibrated ruler/caliper) and record the reference method. Then repeat selected captures with a changed view, partial occlusion and reflective/glare conditions. If labelled weld and non-weld edges are available, use them as ground truth; otherwise report a qualitative candidate review instead of precision/recall.

Test / metric	Calculation or comparison	Evidence to save
Dimension/endpoint error	Absolute length error: $\|L_{est} - L_{ref}\|$. Endpoint error: $\|\|P_{est} - P_{ref}\|\|$ for corresponding points. Report mean, P95 and maximum where enough samples exist.	Reference measurements, estimated coordinates, capture distance, repeats, error CSV.
Point-cloud quality	Valid-depth fraction = valid pixels / region-of-interest pixels; report missing-depth fraction and point count.	RGB image, raw depth, masked depth, point-cloud screenshot and counts.
Surface fit / segmentation	Plane residual RMSE; cluster/part labels compared against an annotated reference.	Segmented cloud and failure examples.
Candidate-joint detection	Precision/recall only if candidate joints and non-weld edges are labelled by a knowledgeable reviewer.	Labelled test set, true/false positives, false negatives.
Repeatability	Spread (e.g., standard deviation) of estimated length and endpoints across repeated captures.	Per-capture outputs and summary table.
Runtime + integration	Processing latency per capture; valid JSON payloads / total payloads; invalid-data rejection test.	Timing log, schema test report and Team 6 parser response.

Acceptance thresholds must be agreed after Team 6 provides the path-planning tolerance and the sensor is characterized. Until then, report measured errors and uncertainty without declaring industrial accuracy. Baseline completion requires repeatable valid output, correct units/frames, successful schema tests and documented failure cases—not an unsupported numeric accuracy claim.

6.2 Risks and limitations

Risk / missing resource	Mitigation / fallback
No metric depth or scanner available	Use 2D RGB for image/edge prototyping only; do not claim reliable metric 3D from a single uncalibrated image.
Reflective steel, glare or welding arc	Capture before arc-on initially; vary viewpoint and exposure; mask invalid depth; flag incomplete regions; validate arc-on separately with safety approval.
Occlusion from clamps/fixtures or unseen joint faces	Take multiple views where feasible; distinguish occluded/unknown from “no joint.”
False candidate at a non-weld edge	Use part drawings/work instructions/operator confirmation; keep weld_intent_confirmed=false until confirmed.
Uncalibrated world/robot transform	Stay in camera_frame or calibrated assembly_world; do not report robot_base coordinates prematurely.

Risk / missing resource	Mitigation / fallback
No labelled sample set or tolerance	Collect annotations and agree measurement limits with project owner/Team 6 before metric claims.
Schema/interface changes	Version the JSON contract and test a valid payload, an invalid payload and a low-confidence payload with Team 6.

Verified facts vs assumptions

Supported by source/documentation	Assumption or project question still open
Open3D documentation describes RANSAC plane segmentation and DBSCAN point-cloud clustering [1]. OpenCV documents camera calibration [2].	These algorithms have not yet been benchmarked on our assembly or reflective weld-cell surfaces.
Manufacturer pages provide nominal depth ranges/specifications for OAK-D Lite and D405 [3][7]; current retailers list indicative prices [4][5][8].	No actual camera has been confirmed available/purchased; real accuracy, work distance, stock and price must be checked.
Team 6's report proposes an input/output contract but explicitly says it needs inter-team agreement.	Final schema, field names, target frame, update rate, tolerances, and responsibility for transforms are not yet approved.
The PCA endpoints/formulas in this report are calculated consistently from the displayed illustrative coordinates.	The example point cloud and values are not real capture data and do not demonstrate performance.

Submission checklist

Requirement group	Where addressed	Status
Objective, current/future scope and sensing alternatives	Section 1	Included
Workflow/architecture and acquisition/calibration	Section 2, Figure 1	Included; proposal, not measured evidence
Classical baseline, algorithms and formulas	Section 3	Included
Technology stack, camera comparison and indicative cost evidence	Section 5	Included; prices are snapshots, not quotes
Input/output JSON, field meanings, units, frames and interfaces	Section 4	Included; schema must be agreed with Team 6
Validation scenarios/metrics, risks and fact/assumption split	Section 6	Included; physical experiment remains to be run
10-day plan, owners and definition of done	Section 7	Included; replace role owners with names

References and supporting sources

Online source prices/specifications were checked on 18 October 2026. Vendor specifications are starting points for selection, not measured performance on this project. Links are provided for verification.

[1] Open3D — Point-cloud processing, RANSAC plane segmentation and DBSCAN clustering. https://www.open3d.org/docs/latest/tutorial/geometry/pointcloud.html

[2] OpenCV — Camera calibration tutorial (intrinsics, distortion and relation between image pixels and real-world units). https://docs.opencv.org/4.12.0/d4/d94/tutorial_camera_calibration.html

[3] Luxonis — OAK-D Lite hardware page and vendor-stated operating range/depth accuracy. https://docs.luxonis.com/hardware/products/OAK-D%20Lite

[4] Robu.in — OAK-D Lite reseller listing; displayed price/availability can change. https://robu.in/product/luxonis-oak-d-lite-robotics-camera-with-auto-focus/

[5] MG Super Labs India — OAK-D Lite listing; verify SKU, stock, tax and shipping. https://www.mgsuperlabs.co.in/estore/OAK-D-Lite

[6] Luxonis Shop — OAK-D Lite official store listing (USD price snapshot). https://shop.luxonis.com/products/oak-d-lite-1

[7] RealSense — D405 official product specifications (7–50 cm ideal range; vendor depth specification). https://www.realsenseai.com/products/stereo-depth-camera-d405/

[8] MG Super Labs India — RealSense D405 and other RealSense product listings; availability may vary. https://www.mgsuperlabs.co.in/estore/Brand/intel-REALSENSE-Products

[9] Tanna TechBiz India — RealSense D405 reseller listing; page displays price/status that may change. https://tannatechbiz.com/realsense-depth-camera-d405.html

[10] Rustler, L., Volprecht, V. & Hoffmann, M. — Empirical Comparison of Four Stereoscopic Depth Sensing Cameras for Robotics Applications (2025 preprint; benchmark is not a test of our exact workcell). https://arxiv.org/abs/2501.07421

[11] Luxonis DepthAI documentation — StereoDepth configuration, alignment, confidence and short-range options. https://docs.luxonis.com/software-v3/depthai/depthai-components/nodes/stereo_depth

[12] Project-internal reference — VENTEK Team 6, Adaptive Path Planning proposal, Sections 8–10: interface contract, stack and validation plan. Supplied by the project team; no public URL.